

CO4_AT3 - Comparative Analysis

Name: C. Himaja

Reg. No: 192373035

Report

Title

Coin Change Problem Comparison: Greedy vs Dynamic Programming

1 Objective

To solve the Coin Change Problem using both the Greedy Algorithm and Dynamic Programming, compare their results, analyze the correctness of both approaches, and study their time and space complexity.

2 Problem Statement

Given the coin denominations:

[1, 4, 6]

and the target amount:

8

the objective is to find the minimum number of coins required to make the target amount.

The problem is solved using:

1. Greedy Algorithm
2. Dynamic Programming

The results are compared to determine which approach produces the optimal solution.

4. Greedy Approach

The Greedy algorithm always chooses the largest denomination possible at each step.

For:

Coins = [1, 4, 6]

Amount = 8

the largest coin that can be selected is 6.

Remaining amount:

$$8 - 6 = 2$$

The next available coin is 1.

Therefore:

$$6 + 1 + 1 = 8$$

Number of coins:

3

Greedy Solution

$$6 \rightarrow 1 \rightarrow 1$$

Total = 3 coins

5. Dynamic Programming Approach

Dynamic Programming considers all possible smaller amounts and stores the minimum number of coins required for each amount.

The recurrence is:

$$dp[amount] = \min(dp[amount - coin] + 1)$$

with:

$$dp[0] = 0$$

DP Table

For coins [1, 4, 6]:

Amount Minimum Coins

0 0

1 1

2 2

3 3

4 1

5 2

6 1

Amount Minimum Coins

7 2

8 2

For amount 8:

$$8 = 4 + 4$$

Therefore:

Minimum coins = 2

6. Comparison

Feature	Greedy	Dynamic Programming
Approach	Select largest coin first	Evaluate all relevant subproblems
Result for 8	3 coins	2 coins
Optimal for this input?	No	Yes
Time Complexity	$O(n + k)$ approximately*	$O(n \times A)$
Space Complexity	$O(n)$	$O(A)$
Implementation	Simple	More complex
Works for all denominations?	No	Yes, when a solution exists

Where:

- n = number of denominations
- A = target amount
- k = number of coins selected by the greedy process

*The exact greedy cost depends on how denominations are processed and how many coins are selected.

7. Why Does Greedy Fail?

The Greedy algorithm makes a locally optimal choice without considering its effect on the remaining amount.

In this example:

Amount = 8

Greedy chooses:

6

because 6 is the largest possible denomination.

This leaves:

$$8 - 6 = 2$$

which requires:

$$1 + 1$$

Therefore:

$$6 + 1 + 1 = 8$$

with 3 coins.

However, a better solution is:

$$4 + 4 = 8$$

with only 2 coins.

Thus:

Greedy = 3 coins

DP = 2 coins

The greedy choice of 6 prevents it from finding the better combination of two 4 coins.

8. When Does Greedy Work?

Greedy works correctly for some coin systems known as canonical coin systems.

For example, common denominations such as:

1, 5, 10, 25

often allow the greedy approach to produce the minimum number of coins.

However, arbitrary denominations may not have this property.

For example:

[1, 4, 6]

is a counterexample.

Therefore, Greedy should not automatically be assumed to produce the optimal solution for every currency system.

9. Dynamic Programming Correctness

Dynamic Programming provides an optimal solution because it examines the best solution for every smaller amount.

For each amount i , the algorithm checks every available coin.

For example, for amount 8:

Using 1 $\rightarrow dp[7] + 1$

Using 4 $\rightarrow dp[4] + 1$

Using 6 $\rightarrow dp[2] + 1$

The minimum is selected.

Since:

$$dp[4] = 1$$

we get:

$$dp[8] = dp[4] + 1$$

$$= 1 + 1$$

$$= 2$$

Thus:

$$4 + 4 = 8$$

is found as the optimal solution.

10. Optimal Substructure

The Coin Change problem has optimal substructure.

The optimal solution for a larger amount can be constructed using optimal solutions to smaller amounts.

For example:

Optimal solution for 8

↓

4 + optimal solution for 4

↓

4 + 4

The optimal solution for the smaller amount 4 is reused to construct the solution for 8.

11. Overlapping Subproblems

The same smaller amounts can occur repeatedly during the calculation.

For example:

dp[4]

dp[2]

dp[7]

may be needed while solving different amounts.

Dynamic Programming stores these results instead of calculating them repeatedly.

This improves efficiency compared with a naive recursive solution.

12. Time Complexity

Let:

- n = number of denominations
- A = target amount

Greedy

The greedy algorithm generally processes the denominations from largest to smallest.

Its cost is approximately:

$O(n + k)$

where k is the number of selected coins.

If the denominations are sorted beforehand, sorting can add:

$O(n \log n)$

if necessary.

Dynamic Programming

For every amount from 1 to A , all n denominations are checked.

Therefore:

Time Complexity = $O(n \times A)$

For:

$n = 3$

$A = 8$

the DP computation is very small.

13. Space Complexity

Greedy

The Greedy algorithm requires very little additional memory.

Space Complexity $\approx O(n)$

for storing the denominations and selection information.

Dynamic Programming

The DP algorithm stores an array for all amounts from 0 to A.

Therefore:

Space Complexity = $O(A)$

The C program also stores the coin used to reconstruct the optimal solution.

14. Scalability

When the number of denominations increases, the behavior of both methods differs.

Greedy

Greedy is computationally lightweight.

For a large number of denominations, it can make decisions quickly, especially if the denominations are already sorted.

However, its main disadvantage remains correctness.

More denominations do not guarantee that greedy will produce the optimal answer.

Dynamic Programming

DP has a time complexity of:

$O(n \times A)$

Therefore, increasing the number of denominations increases the number of calculations.

For example:

10 denominations $\times$ 10,000 amount

requires approximately:

100,000 DP state evaluations

This can still be practical for many applications.

However, very large target amounts can make traditional DP expensive in both execution time and memory.

15. Currency System Insights

The choice between Greedy and Dynamic Programming depends on the properties of the currency system.

Use Greedy when:

- The currency system is known to be canonical.
- Fast computation is important.
- The denomination set is stable.
- A provably optimal greedy strategy exists.
- The application has very large transaction volumes and simple denominations.

Use Dynamic Programming when:

- Denominations are arbitrary.
- Guaranteed minimum coins are required.
- The currency system changes frequently.
- Correctness is more important than minimal computation.
- The denomination system is not known to be canonical.

16. Industry Applications

The Coin Change problem has applications beyond physical currency.

Examples:

- ATM cash dispensing
- Vending machines
- Payment processing
- Digital wallet systems
- Resource allocation
- Token-based systems
- Manufacturing material selection

For example, an ATM may need to dispense a specific amount using the minimum number of notes.

If the available notes are unusual, a Dynamic Programming approach can guarantee an optimal result.

17. Performance Analysis

For the given input:

Denominations = [1, 4, 6]

Amount = 8

Greedy

$6 + 1 + 1$

Number of coins:

3

Dynamic Programming

$4 + 4$

Number of coins:

2

Difference

$3 - 2 = 1$ coin

Therefore, DP saves one coin compared with the greedy solution.

18. Result

The Coin Change problem was successfully implemented in C using Greedy and Dynamic Programming approaches.

For the denomination set:

[1, 4, 6]

and amount:

8

the results are:

Greedy = 3 coins

Dynamic Programming = 2 coins

The Greedy algorithm fails because its locally optimal choice of 6 leads to the non-optimal combination $6 + 1 + 1$.

Dynamic Programming evaluates the possible combinations and correctly finds:

$4 + 4 = 8$

using only 2 coins.

Hence, Dynamic Programming is the preferred approach when guaranteed optimality is required for arbitrary denominations, while Greedy is preferable when the currency system is known to support greedy optimality and speed is the primary concern.

19. Screenshots to Include in Report

This screenshot shows the initial code in `main.c` and the output of the program. The code defines a `greedyCoinChange` function and a `main` function. The output displays the coin denominations [1, 4, 6] and the amount 8. It compares the Greedy method (3 coins) with the Dynamic Programming method (2 coins).

```
main.c
1 #include <stdio.h>
2 #include <limits.h>
3
4 #define MAX 100
5
6 // Function to solve Coin Change using Greedy method
7 int greedyCoinChange(int coins[], int n, int amount, int selected
8 []) {
9     int count = 0;
10    int remaining = amount;
11
12    // Start from the largest denomination
13    for (int i = n - 1; i >= 0; i--) {
14        while (remaining >= coins[i]) {
15            remaining -= coins[i];
16            selected[i]++;
17            count++;
18        }
19    }
20    return count;
21 }
```

```
Output
=====
COIN CHANGE PROBLEM COMPARISON
=====

Coin Denominations: [1, 4, 6]
Amount: 8

----- GREEDY METHOD -----
Coins selected:
2 x 1 coin(s)
1 x 6 coin(s)
Total number of coins = 3

----- DYNAMIC PROGRAMMING METHOD -----
Coins selected: 4 4
Total number of coins = 2

----- COMPARISON -----
```

This screenshot shows the addition of error handling and DP array initialization to the code. The `greedyCoinChange` function is updated to return -1 if the amount cannot be formed. The `dpCoinChange` function is initialized with a DP array of size `amount + 1`.

```
main.c
20 // If amount cannot be formed
21 if (remaining != 0)
22     return -1;
23
24 return count;
25 }
26
27 // Function to solve Coin Change using Dynamic Programming
28 int dpCoinChange(int coins[], int n, int amount, int dp[], int
29 used[]) {
30     // Initialize DP array
31     dp[0] = 0;
32
33     for (int i = 1; i <= amount; i++) {
34         dp[i] = INT_MAX;
35         used[i] = -1;
36     }
37 }
```

```
Output
----- GREEDY METHOD -----
Coins selected:
2 x 1 coin(s)
1 x 6 coin(s)
Total number of coins = 3

----- DYNAMIC PROGRAMMING METHOD -----
Coins selected: 4 4
Total number of coins = 2

----- COMPARISON -----
Greedy Coin Count = 3
DP Coin Count    = 2
Greedy does NOT give the optimal solution.
Dynamic Programming gives the optimal solution.

=====
```

This screenshot shows the completion of the DP algorithm. The `dpCoinChange` function is updated to include a nested loop for all coin denominations, updating the DP array and the `used` array. The `main` function calls `dpCoinChange` and prints the result.

```
main.c
38 int dpCoinChange(int coins[], int n, int amount, int dp[], int
39 used[]) {
40     for (int i = 1; i <= amount; i++) {
41         for (int j = 0; j < n; j++) {
42             if (coins[j] <= i && dp[i - coins[j]] != INT_MAX) {
43                 if (dp[i - coins[j]] + 1 < dp[i]) {
44                     dp[i] = dp[i - coins[j]] + 1;
45                     used[i] = coins[j];
46                 }
47             }
48         }
49     }
50     return dp[amount];
51 }
```

```
Output
----- GREEDY METHOD -----
Coins selected:
2 x 1 coin(s)
1 x 6 coin(s)
Total number of coins = 3

----- DYNAMIC PROGRAMMING METHOD -----
Coins selected: 4 4
Total number of coins = 2

----- COMPARISON -----
Greedy Coin Count = 3
DP Coin Count    = 2
Greedy does NOT give the optimal solution.
Dynamic Programming gives the optimal solution.

=====
```

```
main.c Run Output
57 void displayCoins(int coins[], int n, int selected[]) {
58
59     for (int i = 0; i < n; i++) {
60         if (selected[i] > 0) {
61             printf("%d x %d coin(s)\n",
62                 selected[i],
63                 coins[i]);
64         }
65     }
66 }
67
68 int main() {
69
70     int coins[] = {1, 4, 6};
71     int n = 3;
72     int amount = 8;
73
74     int greedySelected[MAX] = {0};
75
76     // Greedy Method
77     int greedyCoinCount = greedyCoinChange(
78         coins,
79         n,
80         amount,
81         greedySelected);
82
83     // Dynamic Programming Method
84     int dpCoinCount = dpCoinChange(
85         coins,
86         n,
87         amount,
88         dp);
89
90     // Comparison
91     printf("Greedy Coin Count = %d\n", greedyCoinCount);
92     printf("DP Coin Count = %d\n", dpCoinCount);
93     printf("Greedy does NOT give the optimal solution.\n");
94     printf("Dynamic Programming gives the optimal solution.\n");
95
96     // Output
97     displayCoins(coins, n, greedySelected);
98
99     return 0;
100 }
```

Greedy Method
Coins selected:
2 x 1 coin(s)
1 x 6 coin(s)
Total number of coins = 3

----- DYNAMIC PROGRAMMING METHOD -----
Coins selected: 4 4
Total number of coins = 2

----- COMPARISON -----
Greedy Coin Count = 3
DP Coin Count = 2
Greedy does NOT give the optimal solution.
Dynamic Programming gives the optimal solution.

=====
=== Code Execution Successful ===

```
main.c Run Output
68 int main() {
69
70     int used[MAX + 1];
71
72     // Greedy Method
73     int greedyCoinCount = greedyCoinChange(
74         coins,
75         n,
76         amount,
77         greedySelected);
78
79     // Dynamic Programming Method
80     int dpCoinCount = dpCoinChange(
81         coins,
82         n,
83         amount,
84         dp);
85
86     // Comparison
87     printf("Greedy Coin Count = %d\n", greedyCoinCount);
88     printf("DP Coin Count = %d\n", dpCoinCount);
89     printf("Greedy does NOT give the optimal solution.\n");
90     printf("Dynamic Programming gives the optimal solution.\n");
91
92     // Output
93     displayCoins(coins, n, greedySelected);
94
95     return 0;
96 }
```

Greedy Method
Coins selected:
2 x 1 coin(s)
1 x 6 coin(s)
Total number of coins = 3

----- DYNAMIC PROGRAMMING METHOD -----
Coins selected: 4 4
Total number of coins = 2

----- COMPARISON -----
Greedy Coin Count = 3
DP Coin Count = 2
Greedy does NOT give the optimal solution.
Dynamic Programming gives the optimal solution.

=====
=== Code Execution Successful ===

```
main.c Run Output
68 int main() {
69
70     int dpResult = dpCoinChange(
71         coins,
72         n,
73         amount,
74         dp,
75         used);
76
77     // Output
78     printf("\n=====");
79     printf(" COIN CHANGE PROBLEM COMPARISON\n");
80     printf("=====");
81
82     printf("\nCoin Denominations: [1, 4, 6]");
83
84     return 0;
85 }
```

Greedy Method
Coins selected:
2 x 1 coin(s)
1 x 6 coin(s)
Total number of coins = 3

----- DYNAMIC PROGRAMMING METHOD -----
Coins selected: 4 4
Total number of coins = 2

----- COMPARISON -----
Greedy Coin Count = 3
DP Coin Count = 2
Greedy does NOT give the optimal solution.
Dynamic Programming gives the optimal solution.

=====
=== Code Execution Successful ===